Ex: No: 1a	Implementation of lexical analyzer using C programming
Date:	

AIM:

To implement YACC using C programming

ALGORITHM:

Step 1: Start the program.
Step 2: Read the C program from the user and store it in a file named input.
Step 3: Open the files input (read mode), identifier, operators, and specialchar (write mode).
Step 4: Initialize variables to store constants, characters, and indices.
Step 5: Read each character from the input file until end of file.
Step 6: If the character is a digit, form the complete number and store it in an array of constants.
Step 7: If the character is an alphabet, continue reading characters to form a word and store it in the identifier file.
Step 8: If the character is an operator (+, -, *, <, >, /, &, %, ^, =), store it in the operator file.
Step 9: Otherwise, treat the character as a special character and store it in the special character file.
Step 10: Close all the files after processing the input.
Step 11: Print all the constants stored in the array.
Step 12: Read each word from the identifier file and compare it with the list of keywords to determine whether it is a keyword or an identifier.
Step 13: Print all special characters from the special character file.
Step 14: Print all operators from the operator file.
Step 15: Stop the program.

PROGRAM:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
void keyword(char str[]) {
   char keywords[7][10] = {"int","float","char","while","do","for","if"};
   int i, flag = 0;

   for(i = 0; i < 7; i++) {
      if(strcmp(keywords[i], str) == 0) {
         flag = 1;
         break;
      }
   }
   if(flag)
      printf("\n%s is a keyword", str);
   else
      printf("\n%s is an identifier", str);
}
```

```
int main() {
    FILE *f1, *f2, *f3, *f4;
    char c, str[20];
    int num[100], tokenvalue = 0;
    int i = 0, j = 0, k = 0;
    printf("Enter the C program (Press Ctrl+D to stop):\n");
    f1 = fopen("input.txt", "w");
    while((c = getchar()) != EOF)
        putc(c, f1);
    fclose(f1);
    f1 = fopen("input.txt", "r");
    f2 = fopen("identifier.txt", "w");
    f3 = fopen("specialchar.txt", "w");
    f4 = fopen("operators.txt", "w");
    while((c = getc(f1)) != EOF) {
        // Numbers
        if(isdigit(c)) {
            tokenvalue = c - '0';
            c = getc(f1);
            while(isdigit(c)) {
                tokenvalue = tokenvalue * 10 + (c - '0');
                c = getc(f1);
            }
            num[i++] = tokenvalue;
            ungetc(c, f1);
        }
        // Identifiers / Keywords
        else if(isalpha(c)) {
            putc(c, f2);
            c = getc(f1);

            while(isalnum(c) || c == '_' || c == '$') {
                putc(c, f2);
                c = getc(f1);
            }
            putc(' ', f2);
            ungetc(c, f1);
        }
        // Operators
        else if(c=='+' || c=='-' || c=='*' || c=='<' || c=='>' ||
                c=='/' || c=='&' || c=='%' || c=='^' || c=='=') {
            putc(c, f4);
        }

        // Special Characters
        else {
```

```
            putc(c, f3);
        }
    }
    fclose(f1);
    fclose(f2);
    fclose(f3);
    fclose(f4);
    // Print constants
    printf("\nConstants are: ");
    for(j = 0; j < i; j++)
        printf("%d ", num[j]);
    // Identify keywords & identifiers
    f2 = fopen("identifier.txt", "r");
    printf("\nKeywords and Identifiers:");
    while((c = getc(f2)) != EOF) {
        if(c != ' ')
            str[k++] = c;
        else {
            str[k] = '\0';
            keyword(str);
            k = 0;
        }
    }
    fclose(f2);
    // Print special characters
    f3 = fopen("specialchar.txt", "r");
    printf("\nSpecial characters: ");
    while((c = getc(f3)) != EOF)
        printf("%c ", c);
    fclose(f3);
    // Print operators
    f4 = fopen("operators.txt", "r");
    printf("\nOperators: ");
    while((c = getc(f4)) != EOF)
        printf("%c ", c);
    fclose(f4);
    return 0;
}
```

OUTPUT:

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

D:\SKCET\PCD>gcc exp1.c

D:\SKCET\PCD>a.exe
Enter the C program (Press Ctrl+D to stop):
a=b+8*2-c/4
^Z

Constants are: 8 2 4
Keywords and Identifiers:
a is an identifier
b is an identifier
c is an identifier
Special characters:

Operators: = + * - /
D:\SKCET\PCD>
```

RESULT:

Thus, the lexical analyzer using C programming is successfully implemented.

Ex: No:1b	Implementation of Lexical analyzer using LEX
Date:	

AIM:

To implement calculator that takes an expression (with digits, + and *), computes andprints its value, using YACC.

ALGORITHM:

Step 1: Start the program.
Step 2: Initialize counters for characters (c), words (w), lines (l), and spaces (s) to zero.
Step 3: Read the input file using Lex (yyin).
Step 4: Scan the file character by character using lexical rules.
Step 5: If a newline (\n) is found, increment line count and space count.
Step 6: If a space or tab is found, increment space count.
Step 7: If a sequence of characters (word) is found, increment word count and add its length to character count.
Step 8: Continue scanning until end of file.
Step 9: Print the number of spaces, characters, lines, and words.
Step 10: Stop the program.

PROGRAM:

Program for Count the word, character, space and line from the file using LEX:

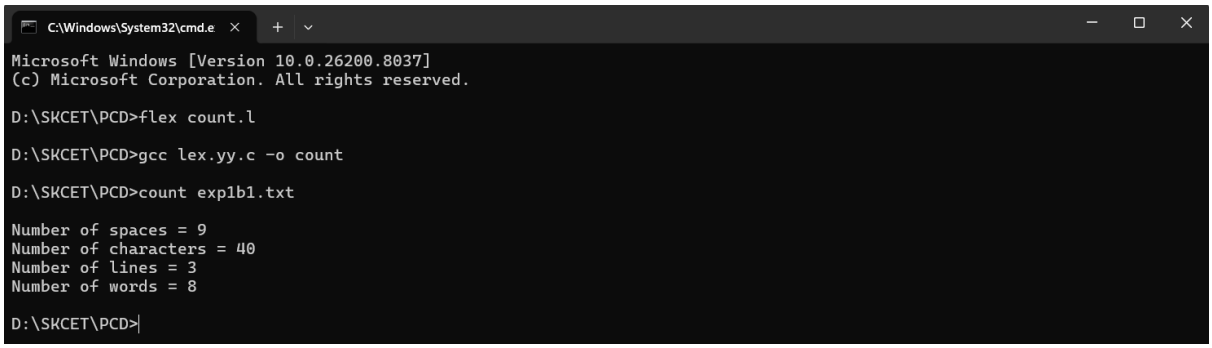
```
%{
int c=0,w=0,l=0,s=0;
%}
%%
[\n] l++;s++;
[\t "] s++;
[^ " \t \n]+ w++;c+=yyleng;
%%
int main(int argc,char *argv[])
{
if(argc==2)
{
}
else
}
yyin=fopen(argv[1],"r");
yylex();
printf("\nNumber of spaces = %d",s);
printf("\nNumber of characters = %d",c);
printf("\nNumber of lines = %d",l);
printf("\nNumber of words = %d\n",w);
printf("ERROR");
int yywrap()
{
```

```
    return 1;
}
```

exp1b1.txt

amazon rainforest
good morning
pcd exp1b of lex

OUTPUT:



PROGRAM for Lexical Analyzer using LEX tool :

```
%{
#include<stdio.h>
%}
letter [a-zA-Z]
digit[0-9]
operators [+*/=%&|<>-]
specialcharacters [();{}"]
%%
(#include<stdio.h>|void|main|int|float|char|printf|while|do|for|if|else|double|break|continue|scan
f|switch|case)+ {printf(" Keyword ");}
{letter}({letter}|{digit})* {printf(" Variable ");}
{digit}+ {printf(" Number ");}
{operators}+ {printf(" Operator ");}
{specialcharacters}+ {printf(" Specialcharacter ");}
%%
int main(int argc,char *argv[])
{
yyin = fopen(argv[1],"r");
yylex();
}
int yywrap()
{
    return 1;
}
```

exp1b2.c

```
#include<stdio.h>
void main()
{
a=b+88;
printf("Welcome to the world of Information Technology");
}
```

OUTPUT:

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

D:\SKCET\PCD>flex lexer.l

D:\SKCET\PCD>gcc lex.yy.c -o lexer

D:\SKCET\PCD>lexer exp1b2.txt
 Keyword
 Keyword   Keyword  Specialcharacter
 Specialcharacter
 Variable  Operator  Variable  Operator  Number  Specialcharacter
 Keyword  Specialcharacter  Variable   Variable   Variable   Variable   Variable   Variable   Variable  Specialcharacter

 Specialcharacter

D:\SKCET\PCD>
```

RESULT:

Thus, the lexical analyzer using C programming and the LEX tool was successfully implemented.

Ex: No:2	Implementation of a calculator that takes an expression (with digits, + and *), computes and prints its value, using YACC.
Date:	

AIM:

To implement calculator that takes an expression (with digits, + and *), computes andprints its value, using YACC.

ALGORITHM:

Step 1: Start the program.
Step 2: Read the arithmetic expression from the user.
Step 3: The lexical analyzer (LEX) scans the input and identifies tokens such as numbers and operators.
Step 4: If a number is found, store its value in yylval and return it as a NUMBER token.
Step 5: Ignore spaces and tabs while scanning.
Step 6: Pass the tokens to the parser (YACC).
Step 7: The parser checks the expression using grammar rules for addition, subtraction, multiplication, division, modulus, and parentheses.
Step 8: Perform corresponding operations based on the rules and evaluate the expression.
Step 9: If the expression follows valid grammar, compute and display the result.
Step 10: If any syntax error occurs, display that the expression is invalid.
Step 11: Print whether the given expression is valid or invalid.
Step 12: Stop the program.

PROGRAM:

LEX Code (calc.l)

```
%{
#include <stdio.h>
#include <stdlib.h>
#include "y.tab.h"
%}
%%
[0-9]+      { yylval = atoi(yytext); return NUMBER; }
[ \t]       ;   // ignore spaces
\n          return 0;
.           return yytext[0];
%%
int yywrap()
{
   return 1;
}
```

YACC Code (calc.y)

```
%{
#include <stdio.h>
#include <stdlib.h>
int flag = 0;
%}
%token NUMBER
```

```
%left '+' '-'
%left '*' '/' '%'
%%
ArithmeticExpression:
   E {
      printf("\nResult = %d\n", $$);
      return 0;
   }
;
E:
     E '+' E { $$ = $1 + $3; }
   | E '-' E { $$ = $1 - $3; }
   | E '*' E { $$ = $1 * $3; }
   | E '/' E { $$ = $1 / $3; }
   | E '%' E { $$ = $1 % $3; }
   | '(' E ')' { $$ = $2; }
   | NUMBER { $$ = $1; }
;
%%
int main()
{
   printf("Enter Arithmetic Expression:\n");
   yyparse();
   if(flag == 0)
      printf("Valid Expression\n");
   return 0;
}
int yyerror()
{
   printf("Invalid Expression\n");
   flag = 1;
   return 0;
}
```

OUTPUT:

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

D:\SKCET\PCD>win_flex cal.l

D:\SKCET\PCD>win_bison -d cal.y

D:\SKCET\PCD>gcc y.tab.c lex.yy.c -o calc

D:\SKCET\PCD>calc

Enter Any Arithmetic Expression :
10+20-30*6

Result=-150

Entered arithmetic expression is Valid


D:\SKCET\PCD>calc

Enter Any Arithmetic Expression :
10*20+5

Result=205

Entered arithmetic expression is Valid


D:\SKCET\PCD>
```

RESULT:

Thus, the arithmetic expression evaluator using LEX and YACC was successfully implemented.

Ex: No:3	Implementation of Parser using LEX and YACC tool
Date:	

AIM:

To implement YACC using C programming.

ALGORITHM:

Step 1: Start the program.
Step 2: Read the input string from the user.
Step 3: The lexical analyzer (LEX) scans the input and identifies tokens.
Step 4: If a number is found, convert it to integer and return it as a NUMBER token.
Step 5: If alphabet characters are found, treat them as NAME tokens.
Step 6: Ignore whitespace such as tabs.
Step 7: For any other characters, return them as tokens.
Step 8: Pass all tokens to the parser (YACC).
Step 9: The parser checks whether the input follows the grammar rules:

- A valid statement can be a NAME, NUMBER, or a list inside parentheses.

Step 10: If the input matches the grammar, print "SUCCESS".
Step 11: If any syntax error occurs, display an error message.
Step 12: Stop the program.

PROGRAM:

LEX Code (lexx1.l)

```
%{
#include "y.tab.h"
#include <stdlib.h>
extern int yylval;
%}
%%
[0-9]+ {
   yylval = atoi(yytext);
   printf("Scanned the number %d\n", yylval);
   return NUMBER;
}
[a-zA-Z]+ {
   printf("Scanned a name\n");
   return NAME;
}
[\t ] {
   printf("Skipped whitespace\n");
}
\n {
   return 0;
}
. {
   printf("Found other data \"%s\"\n", yytext);
```

```
    return yytext[0];
}
%%
int yywrap() {
    return 1;
}
```

YACC Code (yacc1.y)

```
%{
#include <stdio.h>
int yylex(void);
int yyparse(void);
int yyerror(char *);
%}
%token NAME NUMBER
%%
stmt : S { printf("SUCCESS\n"); }
     ;
S : '(' L ')'
  | NAME
  | NUMBER
  ;
L : L ',' S
  | S
  ;
%%
int main(void)
{
    return yyparse();
}
int yyerror(char *msg)
{
    return fprintf(stderr, "YACC: %s\n", msg);
}
```

OUTPUT:

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

D:\SKCET\PCD>yacc -d yacc1.y

D:\SKCET\PCD>win_flex lexx1.l

D:\SKCET\PCD>gcc lex.yy.c y.tab.c -o li

D:\SKCET\PCD>li
((id,id),(id))
Found other data "("
Found other data "("
Scanned a name
Found other data ","
Scanned a name
Found other data ")"
Found other data ","
Found other data "("
Scanned a name
Found other data ")"
Found other data ")"
SUCCESS

D:\SKCET\PCD>
```

PROGRAM 2:

LEX Code (lexx1.l)

```
%{
#include "y.tab.h"
extern int yylval;
%}
%%
[0-9]+ {yylval=atoi(yytext);
printf("Scanned the number %d\n",yylval);
return NUMBER;}
[a-zA-Z]+ {printf("Scanned a name\n");
return NAME;}
[\t] {printf("Skipped whitespace\n");}
\n {return 0;}
. {printf("Found other data \"%s\"\n",yytext);
return yytext[0];
}
%%
int yywrap()
{
   return 1;
}
```

YACC Code (yacc2.y)

```
%{
#include<stdio.h>
int yylex(void);     // from LEX
int yyparse(void);   // from YACC
```

```
int yyerror(char *);
%}
%token NAME NUMBER
%%
stmt:E {printf("SUCCESS\n");}
E : E'+'T{}
|T
;
T : T'*'F{}
|F
;
F : '('E')'{}
|NAME{}
|NUMBER{}
;
%%
int main(void)
{
return yyparse();
}
int yyerror(char *msg)
{
return fprintf(stderr,"YACC:%s\n",msg);
}
```

OUTPUT:

RESULT:

Thus, the LEX and YACC program for recognizing valid statements was successfully implemented.

Ex: No:4	Implementation of Symbol table
Date:	

AIM:

To implement the symbol table using C programming.

ALGORITHM:

Step 1: Start the program.
Step 2: Open the input file (input.txt) in read mode.
Step 3: Initialize variables and a structure array to store variable name, datatype, line number, and value.
Step 4: Read each word from the file until end of file.
Step 5: If the word is a datatype (int, float, or char), store it and increment the line number.
Step 6: Read the variable names one by one until ; is encountered.
Step 7: For each variable:

- Store variable name
- Store datatype
- Store line number

Step 8: Check if the variable has an assigned value (=):

- If yes, read and store the value
- If no, assign default value as "garbage"

Step 9: Continue processing variables separated by commas.
Step 10: Repeat the process for all declarations in the file.
Step 11: After processing, display the symbol table with variable name, datatype, line number, and value.
Step 12: Close the file.
Step 13: Stop the program.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
struct symtab {
   int lineno;
   char var[25], dt[25], val[25];
} sa[20];
int main() {
   FILE *fp;
   char s[25], type[25];
   int i = 0, line = 0;
   fp = fopen("input.txt", "r");
   printf("\nSYMBOL TABLE\n");
   printf("Variable\tDatatype\tLine No\t\tValue\n");

   while (fscanf(fp, "%s", s) != EOF) {
      // Check datatype
      if (strcmp(s, "int") == 0 || strcmp(s, "float") == 0 || strcmp(s, "char") == 0) {
         strcpy(type, s);
```

```
            line++;
            while (fscanf(fp, "%s", s) != EOF && strcmp(s, ";") != 0) {
                // Skip commas
                if (strcmp(s, ",") == 0)
                    continue;
                i++;
                sa[i].lineno = line;
                strcpy(sa[i].var, s);
                strcpy(sa[i].dt, type); // Check for assignment
                fscanf(fp, "%s", s);

                if (strcmp(s, "=") == 0) {
                    fscanf(fp, "%s", s);
                    strcpy(sa[i].val, s);
                } else {
                    strcpy(sa[i].val, "garbage");
                }
            }
        }
    }// Print symbol table
    for (int j = 1; j <= i; j++) {
        printf("%s\t\t%s\t\t%d\t\t%s\n",
               sa[j].var, sa[j].dt, sa[j].lineno, sa[j].val);
    }
    fclose(fp);
    return 0;
}
```

Input File (input.txt)

```
int a , b = 88 ; float z ;
char d = 'k' ;
```

OUTPUT:

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

D:\SKCET\PCD>gcc symbol.c

D:\SKCET\PCD>a.exe

SYMBOL TABLE
Variable        Datatype        Line No         Value
a               int             1               garbage
b               int             1               88
z               float           2               garbage
char            float           2               garbage
=               float           2               garbage

D:\SKCET\PCD>
```

RESULT:

Thus, the symbol table management program was successfully implemented to store variable name, datatype, line number, and value.